EVM 가스 계산 메커니즘 분석

개념적 배경

가스(Gas)는 Ethereum에서 계산 리소스 사용량을 측정하고 제한하는 핵심 메커니즘입니다. 모든 EVM 연산은 가스를 소모하며, 이를 통해 무한 루프를 방지하고 네트워크 리소스를 공정하게 분배합니다.

설계 철학

1. **계산 복잡도 반영**: 연산의 실제 비용을 가스로 표현
2. **DoS 공격 방지**: 과도한 계산을 통한 네트워크 공격 차단
3. **리소스 공정 분배**: 블록 가스 한도를 통한 처리량 제어
4. **경제적 인센티브**: 가스 가격을 통한 우선순위 결정

가스 종류

1. **고정 가스(Constant Gas)**: 연산 자체의 기본 비용
2. **동적 가스(Dynamic Gas)**: 입력 크기나 상태에 따라 변하는 비용
3. **메모리 가스**: 메모리 확장에 따른 추가 비용
4. **스토리지 가스**: 상태 변경에 따른 비용

구현 분석

가스 상수 정의

기본 연산 가스 비용

```go
const (
GasQuickStep  uint64 = 2  // 빠른 연산 (PUSH, DUP, SWAP 등)
GasFastestStep  uint64 = 3  // 가장 빠른 연산 (ADD, SUB, MUL 등)
GasFastishStep  uint64 = 4  // 약간 느린 연산
GasFastStep  uint64 = 5  // 보통 연산 (DIV, MOD 등)
GasMidStep  uint64 = 8  // 중간 연산 (ADDMOD, MULMOD)
GasSlowStep  uint64 = 10  // 느린 연산 (JUMPI)
GasExtStep  uint64 = 20  // 확장 연산 (BLOCKHASH)
)
```

특수 연산 가스 비용

```go
const (
TxGas  uint64 = 21000  // 기본 트랜잭션 가스
TxGasContractCreation  uint64 = 53000  // 컨트랙트 생성 트랜잭션
CallValueTransferGas  uint64 = 9000  // 이더 전송 시 추가 가스
CallNewAccountGas  uint64 = 25000  // 새 계정 생성 시 추가 가스
SstoreSetGas  uint64 = 20000  // 스토리지 슬롯 생성
SstoreResetGas  uint64 = 5000  // 스토리지 슬롯 변경
SstoreClearGas  uint64 = 5000  // 스토리지 슬롯 삭제
SstoreRefundGas  uint64 = 15000  // 스토리지 삭제 시 환불
)
```

메모리 가스 계산

메모리 확장은 이차 함수로 가스를 계산합니다:

```
func memoryGasCost(mem *Memory, newMemSize uint64) (uint64, error) {
if newMemSize == 0 {
return 0, nil
}
// 오버플로우 방지 체크
if newMemSize > 0x1FFFFFFFE0 {
return 0, ErrGasUintOverflow
}
newMemSizeWords := toWordSize(newMemSize) // 32바이트 워드 단위로 변환
newMemSize = newMemSizeWords * 32
if newMemSize > uint64(mem.Len()) {
// 이차 함수: gas = linear_coef + quadratic_coef
square := newMemSizeWords * newMemSizeWords
linCoef := newMemSizeWords * params.MemoryGas // 선형 계수 (3 * words)
quadCoef := square / params.QuadCoeffDiv // 이차 계수 (words² / 512)
newTotalFee := linCoef + quadCoef
fee := newTotalFee - mem.lastGasCost // 증분만 계산
mem.lastGasCost = newTotalFee
return fee, nil
}
```

```
return 0, nil
}
```

메모리 가스 공식

```
memory_gas = 3 * memory_size_words + (memory_size_words)² / 512
```

이 이차 함수는 대용량 메모리 사용을 억제하여 DoS 공격을 방지합니다.

스토리지 가스 계산 (SSTORE)

스토리지 가스는 가장 복잡한 계산 로직을 가집니다. EIP-2200에 따른 계산:

```
func gasSStoreEIP2200(evm *EVM, contract *Contract, stack *Stack, mem *Memory,
// 재진입 공격 방지를 위한 최소 가스 체크
if contract.Gas <= params.SstoreSentryGasEIP2200 {
return 0, errors.New("not enough gas for reentrancy sentry")
}
var (
y, x = stack.Back(1), stack.Back(0)
current, original = evm.StateDB.GetStateAndCommittedState(contract.Address(), x.
)
value := common.Hash(y.Bytes32())
if current == value { // 값이 변경되지 않음 (no-op)
return params.SloadGasEIP2200, nil // 800 가스
}
```

```
if original == current { // 원본과 현재 값이 같음 (clean slot)
if original == (common.Hash{}) { // 0에서 non-0으로 변경
return params.SstoreSetGasEIP2200, nil // 20000 가스
}
if value == (common.Hash{}) { // non-0에서 0으로 변경
evm.StateDB.AddRefund(params.SstoreClearsScheduleRefundEIP2200) // 15000 환불
}
return params.SstoreResetGasEIP2200, nil // 5000 가스
}
// dirty slot 처리
if original != (common.Hash{}) {
if current == (common.Hash{}) { // 0에서 non-0으로 재생성
evm.StateDB.SubRefund(params.SstoreClearsScheduleRefundEIP2200)
} else if value == (common.Hash{}) { // non-0에서 0으로 삭제
evm.StateDB.AddRefund(params.SstoreClearsScheduleRefundEIP2200)
}
}
if original == value { // 원본 값으로 복원
if original == (common.Hash{}) {
evm.StateDB.AddRefund(params.SstoreSetGasEIP2200 - params.SloadGasEIP2200) // 19
} else {
evm.StateDB.AddRefund(params.SstoreResetGasEIP2200 - params.SloadGasEIP2200) //
}
```

```
}
return params.SloadGasEIP2200, nil // 800 가스 (dirty update)
}
```

SSTORE 가스 규칙 요약

상황	가스 비용	환불
No-op (값 변경 없음)	800	0
Clean slot, 0 → non-0	20000	0
Clean slot, non-0 → 0	5000	15000
Clean slot, non-0 → non-0	5000	0
Dirty slot	800	상황에 따라

동적 가스 계산 함수들

메모리 복사 연산 (CALLDATACOPY, CODECOPY 등)

```
func memoryCopierGas(stackpos int) gasFunc {
return func(evm *EVM, contract *Contract, stack *Stack, mem *Memory, memorySize
// 메모리 확장 가스
gas, err := memoryGasCost(mem, memorySize)
if err != nil {
return 0, err
}
```

```
// 복사할 데이터 크기 (워드 단위)
words, overflow := stack.Back(stackpos).Uint64WithOverflow()
if  overflow {
return  0, ErrGasUintOverflow
}
// 복사 가스: 3 가스 per 워드
if  words, overflow = math.SafeMul(toWordSize(words), params.CopyGas);  overflow
return  0, ErrGasUintOverflow
}
if  gas, overflow = math.SafeAdd(gas, words);  overflow {
return  0, ErrGasUintOverflow
}
return  gas, nil
}
}
```

Keccak256 해시 계산

```
func  gasKeccak256(evm *EVM, contract *Contract, stack *Stack, mem *Memory, memo
gas, err := memoryGasCost(mem, memorySize)
if  err != nil {
return  0, err
}
```

```
// 해시할 데이터 크기
wordGas, overflow := stack.Back(1).Uint64WithOverflow()
if  overflow {
return  0, ErrGasUintOverflow
}
// Keccak256 가스: 30 + 6 * words
if  wordGas, overflow = math.SafeMul(toWordSize(wordGas), params.Keccak256WordGa
return  0, ErrGasUintOverflow
}
if  gas, overflow = math.SafeAdd(gas, wordGas);  overflow {
return  0, ErrGasUintOverflow
}
return  gas, nil
}
```

EXP 지수 연산

```
func  gasExpEIP158(evm *EVM, contract *Contract, stack *Stack, mem *Memory, memo
// 지수의 바이트 길이 계산
expByteLen := uint64((stack.data[stack.len()-2].BitLen() + 7) / 8)
var (
gas = expByteLen * params.ExpByteEIP158  // 50 가스 per 바이트
overflow  bool
```

```
)
if gas, overflow = math.SafeAdd(gas, params.ExpGas); overflow { // 기본 10 가스
return 0, ErrGasUintOverflow
}
return gas, nil
}
```

CALL 계열 연산 가스

CALL, CALLCODE, DELEGATECALL, STATICCALL의 가스 계산:

```
func gasCall(evm *EVM, contract *Contract, stack *Stack, mem *Memory, memorySiz
var (
gas uint64
transfersValue = !stack.Back(2).IsZero() // 이더 전송 여부
address = common.Address(stack.Back(1).Bytes20())
)
// 새 계정 생성 시 추가 가스
if evm.chainRules.IsEIP158 {
if transfersValue && evm.StateDB.Empty(address) {
gas += params.CallNewAccountGas // 25000 가스
}
} else if !evm.StateDB.Exist(address) {
```

```
gas += params.CallNewAccountGas

}

// 이더 전송 시 추가 가스

if transfersValue && !evm.chainRules.IsEIP4762 {

gas += params.CallValueTransferGas // 9000 가스

}

// 메모리 확장 가스

memoryGas, err := memoryGasCost(mem, memorySize)

if err != nil {

return 0, err

}

var overflow bool

if gas, overflow = math.SafeAdd(gas, memoryGas); overflow {

return 0, ErrGasUintOverflow

}

// EIP-150: 63/64 규칙 적용

evm.callGasTemp, err = callGas(evm.chainRules.IsEIP150, contract.Gas, gas, stack

if err != nil {

return 0, err

}

if gas, overflow = math.SafeAdd(gas, evm.callGasTemp); overflow {

return 0, ErrGasUintOverflow

}
```

```
return gas, nil
}
```

EIP-150 63/64 규칙

```
func callGas(isEip150 bool, availableGas, base uint64, callCost *uint256.Int)
if isEip150 {
availableGas = availableGas - base
gas := availableGas - availableGas/64 // 63/64만 전달 가능
if !callCost.IsUint64() || gas < callCost.Uint64() {
return gas, nil
}
}
if !callCost.IsUint64() {
return 0, ErrGasUintOverflow
}
return callCost.Uint64(), nil
}
```

EIP별 가스 비용 변경사항

EIP-150 (Tangerine Whistle)

- BALANCE: 20 → 400 가스

- EXTCODESIZE: 20 → 700 가스
- SLOAD: 50 → 200 가스
- EXTCODECOPY: 20 → 700 가스 (기본)
- CALL 계열: 40 → 700 가스

EIP-158 (Spurious Dragon)

- EXP: 10 + 10*bytes* → *10 + 50*bytes

EIP-1884 (Istanbul)

- BALANCE: 400 → 700 가스
- EXTCODEHASH: 400 → 700 가스
- SLOAD: 200 → 800 가스

EIP-2200 (Istanbul)

- SSTORE 가스 계산 로직 완전 개편
- 재진입 공격 방지를 위한 2300 가스 센트리

EIP-2929 (Berlin)

- 콜드 계정 접근: 2600 가스
- 콜드 스토리지 접근: 2100 가스
- 웜 스토리지 접근: 100 가스

EIP-3529 (London)

- 환불 한도: 50% → 20%로 축소
- SELFDESTRUCT 환불 제거

EIP-3860 (Shanghai)

- 초기 코드 크기 제한: 49152 바이트
- 초기 코드 가스: 2 가스 per 워드

최적화 전략

1. 가스 효율적인 코드 작성

스토리지 최적화

```
// 비효율적: 여러 번의 SSTORE
contract Inefficient {
uint256 a;
uint256 b;
function  update(uint256 _a, uint256 _b) external {
a = _a;  // 20000 가스 (첫 번째 설정)
b = _b;  // 20000 가스 (첫 번째 설정)
}
}

// 효율적: 패킹을 통한 단일 SSTORE
contract Efficient {
uint256 packed;  // a (128비트) + b (128비트)
function  update(uint128 _a, uint128 _b) external {
packed = (uint256(_a) << 128) | uint256(_b);  // 20000 가스 (한 번만)
```

```
}

}
```

메모리 최적화

```
// 비효율적: 대용량 메모리 할당

function  inefficient(bytes calldata  data) external {

bytes memory temp = new bytes(data.length * 1000);  // 이차 함수 가스

}


// 효율적: 필요한 만큼만 할당

function  efficient(bytes calldata  data) external {

bytes memory temp = new bytes(data.length);  // 선형 가스

}
```

2. 가스 추정 최적화

정확한 가스 한도 설정

```
// 가스 추정 시 여유분 추가

estimatedGas := core.IntrinsicGas(data, accessList, isContractCreation, isHomest

gasLimit := estimatedGas * 110 / 100  // 10% 여유분
```

가스 가격 최적화

```
// EIP-1559 환경에서 가스 가격 설정

baseFee := block.BaseFee()

maxPriorityFeePerGas := big.NewInt(2000000000) // 2 Gwei

maxFeePerGas := new(big.Int).Add(

new(big.Int).Mul(baseFee, big.NewInt(2)), // baseFee * 2

maxPriorityFeePerGas,

)
```

실제 사용 예시

간단한 전송 트랜잭션

```
기본 트랜잭션 가스: 21000

데이터 가스 (0바이트): 0

총 가스: 21000
```

컨트랙트 호출

```
기본 트랜잭션 가스: 21000

CALL 가스: 700 (EIP-150 이후)

이더 전송 가스: 9000 (값 전송 시)

메모리 확장 가스: 가변

실행 가스: 가변
```

```
총 가스: 30700 + 실행 가스
```

스토리지 변경

```
SSTORE (0 → non-0): 20000 가스

SSTORE (non-0 → non-0): 5000 가스

SSTORE (non-0 → 0): 5000 가스 - 15000 환불
```

에러 처리

가스 부족 에러

```
if contract.Gas < cost {
return nil, ErrOutOfGas
}
```

오버플로우 방지

```
if gas, overflow = math.SafeAdd(gas, additionalGas); overflow {
return 0, ErrGasUintOverflow
}
```

환불 한도 체크

```
refund := state.GetRefund()

maxRefund := gasUsed / params.RefundQuotientEIP3529  // 20%

if  refund > maxRefund {

refund = maxRefund

}
```

관련 시스템

의존성 관계

- **StateDB**: 스토리지 상태 조회
- **Memory**: 메모리 크기 추적
- **Stack**: 연산 매개변수 제공
- **ChainRules**: 하드포크별 규칙 적용

상호작용 패턴

1. **Interpreter → Gas Calculator**: 각 opcode 실행 전 가스 계산
2. **Gas Calculator → StateDB**: 스토리지 상태 조회
3. **Gas Calculator → Memory**: 메모리 확장 비용 계산
4. **Gas Calculator → Contract**: 가스 차감

영향 범위

- **트랜잭션 실행**: 모든 연산의 가스 소모
- **블록 가스 한도**: 블록당 처리 가능한 트랜잭션 수

- **가스 가격**: 트랜잭션 우선순위 결정
- **경제적 보안**: DoS 공격 방지

이 가스 계산 메커니즘은 Ethereum 네트워크의 안정성과 공정성을 보장하는 핵심 시스템으로, 복잡하지만 정교하게 설계된 경제적 인센티브 구조를 제공합니다.